

HARNESS ENGINEERING

하네스 엔지니어링

AI 에이전트 시대의 새로운 엔지니어링 패러다임

왕초보부터 실무 적용까지 — 완전 가이드북

2026년 3월 | 에이나우(AINOW)

Prompt Engineering → Context Engineering → Harness Engineering

OpenAI · Anthropic · Mitchell Hashimoto · Martin Fowler 등 공개 자료 기반

목차 (Table of Contents)

PART 1

기초 이해: 하네스 엔지니어링이란 무엇인가

Chapter 1. 하네스 엔지니어링의 탄생

1.1 용어의 기원

2026년 2월, HashiCorp의 공동창업자이자 Terraform의 창시자인 Mitchell Hashimoto가 자신의 블로그에 'My AI Adoption Journey'라는 글을 발표하며 하나의 개념을 명확하게 정의했습니다.

"*Harness Engineering*이란, AI 에이전트가 실수를 할 때마다 그 실수를 다시는 반복하지 못하도록 해결책을 설계하는 것이다." — *Mitchell Hashimoto*,
2026.02.05

몇일 후, OpenAI가 'Harness engineering: leveraging Codex in an agent-first world'라는 제목의 상세 보고서를 발표하며 이 용어는 급속도로 업계 전반에 퍼졌습니다. OpenAI 내부 팀이 5개월에 걸쳐 100만 줄의 코드를 생성했는데, 인간이 직접 작성한 코드는 단 한 줄도 없었습니다.

1.2 왜 지금 하네스 엔지니어링인가?

하네스 엔지니어링이 등장한 배경에는 3가지 수렴 요인이 있습니다:

1. 모델의 상품화(Commoditization): Claude, GPT-4, Gemini 등 최상위 모델들 간의 성능 차이가 줄어들면서, 모델 자체보다 모델을 둘러싼 시스템이 경쟁력을 결정하게 됨
2. 에이전트의 프로덕션 전환: 2025년에는 대부분의 에이전트가 데모 수준이었지만, 2026년부터 실제 프로덕션 환경에 투입되기 시작
3. 신뢰성 병목 현상: 에이전트가 충분히 똑똑해져서 유용하게 되었지만, 실제 시스템에서 안정적으로 동작할 만큼 신뢰할 수 없는 상태

1.3 하네스(Harness)의 비유

'Harness'는 원래 말을 부리기 위해 사용하는 마구(馬具), 즉 고삐와 고삐, 재갈 등을 의미합니다. 강력하지만 예측할 수 없는 말의 힘을 원하는 방향으로 전달하는 장치입니다.

AI 맥락에서도 똑같습니다. AI 모델(말)은 강력하지만 스스로 방향을 정하지 못합니다. 하네스(마구)는 이 힘을 안전하고 안정적으로 방향 지을 수 있게 해주는 시스템입니다.

1.4 컴퓨터 비유로 이해하기

Philipp Schmid(Hugging Face/AWS)는 컴퓨터 비유를 통해 하네스의 위치를 직관적으로 설명했습니다:

컴퓨터 구성요소	AI 시스템 대응
CPU (중앙처리장치)	AI 모델 (Claude, GPT 등)
RAM (메모리)	컨텍스트 윈도우 (Context Window)
운영체제 (OS)	하네스 (Harness)
애플리케이션	에이전트 (Agent)

CPU가 아무리 빠라도 운영체제 없이는 쓸모없듯이, AI 모델이 아무리 똑똑해도 하네스 없이는 실제 업무에서 안정적으로 작동할 수 없습니다.

Chapter 2. 3단계 진화: Prompt → Context → Harness

2.1 프롬프트 엔지니어링 (2022–2024)

첫 번째 물결. AI와의 상호작용이 단일 입력-출력 구조였던 시기입니다. 한 번의 질문에 최적의 답변을 받기 위해 프롬프트를 정교하게 다듬는 기술이었습니다.

- **Few-shot 학습:** 예시를 몇 개 포함하여 모델 유도
- **Chain-of-Thought:** 단계별 사고를 유도하여 추론 능력 향상
- **Role-playing:** 모델에게 역할을 부여하여 문맥 설정

한계: 한 번의 대화로 모든 것을 해결해야 했으며, 복잡한 작업이나 장기 프로젝트에는 부적합했습니다.

2.2 컨텍스트 엔지니어링 (2025)

Andrej Karpathy의 언급으로 널리 알려진 개념입니다. 단일 프롬프트가 아니라, 모델이 정보에 기반한 결정을 내릴 수 있도록 문맥 전체를 설계하는 것입니다.

- **RAG (Retrieval-Augmented Generation):** 외부 문서 검색 + 모델 결합
- **MCP (Model Context Protocol):** 외부 서비스와 모델 연결 표준화
- **메모리 시스템:** 대화 간 문맥 유지
- **Tool 사용:** 모델이 외부 도구를 직접 호출

한계: 문맥을 잘 구성해도 에이전트가 장기간 안정적으로 작동하는 것을 보장할 수 없었습니다.

2.3 하네스 엔지니어링 (2026~)

프롬프트가 '무엇을 물어볼 것인가'이고, 컨텍스트가 '무엇을 보여줄 것인가'라면, 하네스는 '전체 시스템이 어떻게 작동하는가'입니다.

"프롬프트 엔지니어링은 어떻게 물어볼지(*what to ask*), 컨텍스트 엔지니어링은 무엇을 보낼지(*what to send*), 하네스 엔지니어링은 전체가 어떻게 작동하는지(*how it operates*)에 대한 것이다." — *Louis Bouchard*

구분	프롬프트 엔지니어링	컨텍스트 엔지니어링
시기	2022–2024	2025
초점	입력 최적화	문맥 설계
범위	단일 프롬프트	RAG, MCP, 메모리

한계	복잡한 작업 불가	장기 안정성 부족
구분	하네스 엔지니어링	비고
시기	2026~	현재 진행 중
초점	환경 설계	시스템 전체
범위	통, 관찰, 피드백 루프	포괄적
한계	엄가기에 대한 연구 필요	진화 중

Chapter 3. 하네스의 핵심 구성요소

하네스는 단일 기술이 아니라 여러 메커니즘의 조합입니다. Martin Fowler(Thoughtworks)의 분석과 OpenAI, Anthropic의 실험 결과를 종합하면 하네스는 크게 5가지 핵심 구성요소로 나뉩니다.

3.1 컨텍스트 엔지니어링 (Context Engineering)

에이전트가 올바른 결정을 내리기 위해 필요한 정보를 적절한 시점에 제공하는 것입니다. CLAUDE.md, AGENTS.md 같은 프로젝트 설명 파일, 아키텍처 문서, 실행 계획 등이 포함됩니다.

OpenAI Codex 팀은 작은 AGENTS.md 하나에서 출발해, 점차 더 깊은 정보 출처로 연결하는 구조를 만들었습니다.

3.2 건축적 제약조건 (Architectural Constraints)

에이전트가 잘못된 행동을 애초에 할 수 없게 막는 구조적 장치입니다.

- 의존성 방향 규칙: domain → application → infrastructure 처럼 모듈 간 의존성 방향을 강제
- 커스텀 린터(Linter): LLM이 아닌 결정론적 도구로 코드 품질을 검증
- 톨 접근 제어: 에이전트가 사용할 수 있는 도구, 수정할 수 있는 파일, 승인 필요한 작업을 정의
- CI/CD 파이프라인: 모든 변경이 자동화된 품질 검사를 통과해야 함

3.3 피드백 루프 (Feedback Loops)

에이전트의 출력을 검증하고, 문제가 있으면 자동으로 수정하게 하는 순환 구조입니다.

Anthropic 연구에서 발견한 핵심 인사이트 중 하나는 'AI 모델은 자신의 작업을 스스로 평가할 때 너무 관대하게 평가하는 경향이 있다'는 것입니다.

이 문제의 해결책으로 Anthropic은 GAN에서 영감을 받은 Generator-Evaluator 분리 구조를 제안했습니다. 코드를 생성하는 에이전트와 평가하는 에이전트를 분리하면, 평가자를 더 엄격하게 튜닝하는 것이 훨씬 더 효과적입니다.

3.4 관찰 및 모니터링 (Observability)

에이전트의 모든 행동을 기록하고, 실패 패턴을 분석하고, 인간이 개입할 수 있는 시점을 파악하는 시스템입니다.

3.5 인간 개입 지점 (Human-in-the-Loop)

권한 모델을 통해 에이전트의 자율성 범위를 정하고, 고위험 결정에서는 반드시 인간의 확인을 받는 구조입니다. **Claude Code**의 기본 권한 모델이 대표적인 예시입니다.

Chapter 4. 핵심 원칙: 하네스 엔지니어링 7가지 법칙

공개된 사례와 연구를 종합하면 하네스 엔지니어링에는 7가지 핵심 원칙이 있습니다.

법칙 1: 환경을 설계하라, 코드를 작성하지 마라

OpenAI Codex 팀의 핵심 교훈. 엔지니어의 역할은 코드를 쓰는 것이 아니라, 에이전트가 안전하게 코드를 쓸 수 있는 환경을 만드는 것.

법칙 2: 실수에서 배우고 영구적으로 방지하라

Hashimoto의 핵심 정의. 에이전트의 실패 → 원인 분석 → 구조적 해결책 → CLAUDE.md에 반영.

법칙 3: 제약이 곧 생산성이다

LangChain의 실험: 모델을 바꾸지 않고 하네스만 개선했더니 15개 LLM의 코딩 성능이 모두 향상.

법칙 4: 생성과 평가를 분리하라

Anthropic의 핵심 발견. 에이전트는 자신의 작업을 객관적으로 평가하지 못하므로, 별도의 평가 에이전트를 둔다.

법칙 5: 탈부착 가능하게 구축하라 (Rippable)

Vercel은 에이전트 툴의 80%를 제거하고 오히려 성능이 향상. 모델이 업그레이드되면 어제의 복잡한 로직이 오늘은 불필요해진다.

법칙 6: 성공은 조용히, 실패만 크게

HumanLayer 팀의 교훈. 테스트 전체 결과(4000줄)를 컨텍스트에 넣으면 에이전트가 혼란에 빠진다. 성공은 삼키고 오류만 표시.

법칙 7: 계획과 실행을 분리하라

Cloudflare의 Boris Tane: '계획 승인 전에 절대 코드를 쓰게 하지 마라.' 계획-검토-실행의 3단계.

PART 2

실제 사례와 적용 방법

Chapter 5. 사례 1: OpenAI Codex — 100만 줄의 하네스

5.1 실험 개요

OpenAI 내부 팀은 2025년 8월부터 5개월간, 인간이 직접 작성한 코드 없이 Codex 에이전트만으로 제품을 개발했습니다. 약 1,500개의 Pull Request를 통해 100만 줄 이상의 코드가 생성되었으며, 수동 개발 대비 약 10배 빠른 속도로 완성되었습니다.

5.2 하네스의 3가지 핵심 계층

(1) 컨텍스트 엔지니어링

코드베이스 안에 지속적으로 확장되는 지식 베이스를 구축했습니다. AGENTS.md가 핵심이지만, 이것이 모든 것을 담지 않고 더 깊은 설계 문서, 아키텍처 맵, 실행 계획으로 연결되는 구조를 활용했습니다.

"에이전트 관점에서, 인컨텍스트(*in-context*)로 접근할 수 없는 정보는 존재하지 않는 것과 같다. Slack 대화, 구글 문서, 사람들의 머릿속에만 있는 정보는 에이전트가 접근할 수 없다." — OpenAI

(2) 건축적 제약조건

엄격한 계층형 아키텍처를 강제했습니다. 각 도메인의 코드는 엄격한 의존성과 인터페이스를 가졌으며, LLM 기반 에이전트뿐 아니라 결정론적 커스텀 린터와 구조 테스트로도 감시했습니다.

(3) 가비지 컬렉션 (Garbage Collection)

백그라운드에서 주기적으로 실행되는 에이전트가 문서의 비일관성이나 아키텍처 규칙 위반을 찾아내고, 자동으로 수정 PR을 열었습니다. Martin Fowler는 이를 '엔트로피(entropy)와 부패(decay)에 대항하는 메커니즘'이라고 표현했습니다.

5.3 핵심 성과 요약

항목	결과
코드 규모	약 100만 줄 (1M+ lines)

인간 작성 코드	0줄 (zero hand-written code)
개발 속도	수동 대비 약 10배
초기 팀 규모	3명의 엔지니어
엔지니어당 일일 PR	평균 3.5개
개발 기간	5개월 (2025.08 ~ 2026.01)

Chapter 6. 사례 2: Anthropic — 장기 실행 에이전트 하네스

6.1 문제 인식

Anthropic은 AI 코딩 에이전트가 복잡한 프로젝트를 수행할 때 발생하는 2가지 핵심 실패 패턴을 발견했습니다:

4. 원샷 시도(One-shotting): 한 번에 모든 것을 해결하려고 함 → 불완전한 결과
5. 조기 종료 선언: 작업을 너무 일찍 완료했다고 판단함

6.2 2단계 에이전트 하네스

Initializer Agent (초기화 에이전트)

첫 번째 세션에서만 실행되며, 프로젝트의 기반 환경을 설정합니다:

- `init.sh`: 개발 서버 시작 및 애플리케이션 실행 방법 정의
- `claude-progress.txt`: 세션 간 작업 진행 상황 기록
- 기능 목록 파일: 사용자의 간단한 요청을 수백 개의 구체적 요구사항으로 확장
- 초기 Git 커밋: 버전 관리 기준점 설정

Coding Agent (코딩 에이전트)

여러 세션에 걸쳐 점진적으로 기능을 구현하며, 매 세션의 시작 시 `claude-progress.txt`와 `git` 이력을 통해 이전 작업 상황을 파악합니다.

6.3 컨텍스트 불안(Context Anxiety) 해결

Anthropic 연구팀은 '컨텍스트 불안'이라는 현상을 발견했습니다. 에이전트가 컨텍스트 윈도우가 차면 작업 품질이 떨어지는 현상입니다. 단순한 컨텍스트 압축(compaction)으로는 해결이 불충분했으며, 컨텍스트 리셋(context reset)이 필요했습니다.

리셋은 에이전트에게 깨끗한 시작점을 제공하되, 인수인계 문서(handoff artifact)가 충분한 상태를 담고 있어야 합니다. 이것이 `claude-progress.txt`의 역할입니다.

6.4 Generator-Evaluator 분리 모델

Anthropic의 주요 발견: 모델은 자신이 생성한 결과를 객관적으로 평가할 수 없습니다. 해결책으로 두 에이전트 분리 구조를 도입했습니다:

- Generator Agent: 코드, 디자인, 콘텐츠 생성 담당

- **Evaluator Agent:** 별도의 평가 기준을 가지고 결과물 검증

이 방식으로 프론트엔드 디자인의 품질을 크게 높일 수 있었으며, 4가지 평가 기준(디자인 품질, 독창성, UX 폭리식, 코드 품질)을 적용했습니다.

Chapter 7. 사례 3: LangChain — 하네스만 바꾸면 15개 LLM이 모두 향상된다

LangChain 팀은 '하네스만 개선하고 모델은 그대로' 실험을 통해 중요한 결과를 발표했습니다. 같은 모델에 하네스만 바꾸니, 코딩 벤치마크에서 평범하던 모델이 최상위권으로 올라갔습니다.

"모델을 바꾸지 않고 하네스만 바꾸면 에이전트를 평범한 수준에서 최상위권으로 끌어올릴 수 있다." — *LangChain*

이 실험의 핵심 시사점은 명확합니다: 경쟁력은 모델 선택에서 오는 것이 아니라, 모델을 둘러싼 시스템 설계에서 옵니다.

Chapter 8. 사례 4: Stripe, Cloudflare 등 기업 적용 사례

8.1 Stripe — Devbox 격리 방식

Stripe의 AI 에이전트 'Minions'는 사람 엔지니어와 동일한 개발 환경(devbox)에서 작동하되, 프로덕션과 인터넷에서 격리된 샌드박스 환경에서 실행됩니다. 400개 이상의 내부 도구를 MCP 서버로 연결했습니다.

"에이전트는 인간 엔지니어와 동일한 컨텍스트와 도구가 필요하다. 뒤늦게 붙인 통합이 아니라, 처음부터 함께 설계된 환경이어야 한다." — *Greg Brockman, Stripe*

8.2 Cloudflare — 계획-실행 분리

Cloudflare의 Boris Tane은 AI 코딩에서 가장 중요한 원칙으로 '계획과 실행의 분리'를 꼽았습니다. 에이전트가 작성한 계획을 인간이 검토·승인한 후에만 실행을 허락합니다.

8.3 Manus — 하네스 버전 업그레이드

Manus 팀은 하네스 V1에서 V2로 전면 재작성하며 수천 시간의 엔지니어링을 투입했습니다. 이 투자가 모델 개선만으로는 도달할 수 없는 경쟁력을 만들었습니다.

PART 3

도구와 플랫폼 가이드

Chapter 9. Claude Code에서의 하네스 엔지니어링

Claude Code는 Anthropic이 제공하는 에이전트 코딩 도구로, 하네스 엔지니어링의 핵심 구성요소를 내장하고 있습니다.

9.1 CLAUDE.md — 하네스의 핵심

CLAUDE.md는 프로젝트 루트에 위치하는 마크다운 파일로, Claude Code가 모든 세션 시작 시 자동으로 읽는 프로젝트 설명서입니다.

효과적인 **CLAUDE.md** 구조

```
# 프로젝트 개요

이 프로젝트는 Next.js 14 기반의 AI 교육 플랫폼입니다.

## 빌드 및 실행

- npm run dev 로 개발 서버 시작
- npm run test 로 테스트 실행

## 코딩 규칙

- 패키지 의존성 방향: domain → application → infrastructure
- 컴포넌트는 TypeScript로 작성
- 커밋 메시지는 한국어로 작성

## 주의사항

- .eslinttrc 수정 금지 (린트 오류는 코드를 수정해서 해결)
- 테스트 없이 PR 생성 금지
```

Hashimoto의 방법론 그대로입니다: 에이전트가 실수할 때마다, 그 실수를 방지하는 규칙을 CLAUDE.md에 추가합니다. 작게 시작하고, 점진적으로 성장시키는 것이 핵심입니다.

9.2 Hooks — 에이전트 생명주기 제어

Hooks는 Claude Code의 행동 전후에 자동으로 실행되는 스크립트입니다:

- **Pre-tool hooks:** 도구 실행 전 자동 검사 (보안 스캔, 권한 확인)
- **Post-tool hooks:** 파일 수정 후 자동 포매팅, 린트 실행
- **Stop hooks:** 에이전트 종료 전 최종 검증 (코드 커버리지 확인 등)

HumanLayer 팀의 실전 팁: **Stop hook**에서 코드 커버리지가 하락한 경우 에이전트가 자동으로 커버리지를 높이도록 유도합니다.

9.3 권한 모델 (Permission Model)

Claude Code는 기본적으로 읽기 전용(read-only) 모드입니다. 파일 수정, 명령어 실행 등은 사용자의 명시적 승인이 필요합니다. 모든 파일 수정은 자동 스냅샷을 통해 복원 가능합니다.

9.4 Skills — 점진적 정보 공개

Skills는 Anthropic이 도입한 개념으로, 필요한 시점에 필요한 지침만 에이전트에게 제공하는 'Progressive Disclosure' 방식입니다. 모든 지침을 시스템 프롬프트에 넣으면 컨텍스트가 폭발하므로, 에이전트가 필요할 때만 선택적으로 로딩합니다.

9.5 서브 에이전트 (Sub-agents)

복잡한 작업을 여러 개의 격리된 컨텍스트 윈도우에서 병렬로 처리할 수 있습니다.

HumanLayer는 서브 에이전트를 'Context Firewall'로 비유합니다: 각 하위 작업이 독립된 컨텍스트에서 실행되어, 중간 노이즈가 상위 오케스트레이션 쿼리에 누적되지 않습니다.

9.6 MCP (Model Context Protocol) 통합

MCP는 에이전트가 외부 서비스와 상호작용할 수 있게 해주는 표준 프로토콜입니다. Claude Code에서 MCP 서버를 연결하면 GitHub, Slack, Google Drive, Notion 등 외부 서비스를 에이전트가 직접 사용할 수 있습니다.

다만 HumanLayer 팀의 실전 경험에 따르면, CLI로 충분히 처리할 수 있는 작업(GitHub, Docker, DB 등)에는 MCP 대신 CLI를 사용하는 것이 더 효과적입니다. 모델이 학습 데이터에서 이미 CLI 사용법을 잘 알고 있기 때문입니다.

Chapter 10. 다양한 하네스 툴 비교

하네스 엔지니어링 생태계는 빠르게 성숙하고 있습니다. 주요 도구를 비교해보겠습니다:

도구	하네스 방식	핵심 특징
Claude Code	CLAUDE.md + Hooks + Permissions	권한 모델, 자동 스냅샷, Skills
OpenAI Codex	AGENTS.md + CI 통합	격리 실행, 커스텀 린터
Cursor	.cursor/rules 파일	파일 패턴별 규칙, 루프 감지
LangGraph	그래프 기반 오케스트레이션	체크포인트 기반 복구
CrewAI	멀티 에이전트 + Flows	역할 분담, 이벤트 드리븐

Chapter 11. AI OS 관점에서의 하네스

11.1 AI OS란 무엇인가

AI OS는 AI 에이전트가 기존 운영체제처럼 작업을 오케스트레이션하고, 의사결정을 내리고, 상호작용을 관리하는 환경을 의미합니다. 하네스는 이 AI OS의 핵심 계층입니다.

Philipp Schmid의 비유를 확장하면:

- **Kernel (OS 핵심):** AI 모델의 추론 엔진
- **Process Manager:** 하네스가 에이전트의 생명주기를 관리
- **File System:** 컨텍스트 엔지니어링이 정보 접근을 관리
- **Permission System:** 권한 모델이 접근 제어를 담당
- **Device Drivers:** MCP 서버가 외부 서비스와 연결

11.2 비개발자를 위한 AI OS 활용

하네스 엔지니어링은 개발자만의 영역이 아닙니다. AI OS 관점에서 비개발자도 하네스의 핵심 원리를 적용할 수 있습니다:

- **컨텍스트 설계:** AI에게 충분한 배경 정보를 구조화하여 제공
- **규칙 문서:** 반복되는 실수를 방지하는 명확한 지침 작성
- **피드백 루프:** AI 결과물을 검증하고 개선하는 반복 수행
- **세션 관리:** 복잡한 작업을 단계별로 분해하여 처리

PART 4

실전 적용 가이드

Chapter 12. 첫 번째 하네스 만들기: 스타터 가이드

Step 1: CLAUDE.md 또는 AGENTS.md 작성하기

프로젝트 루트에 설명 파일을 만듭니다. 처음부터 완벽하게 쓸 필요 없습니다. 기본 구조만 잡고 시작하세요:

```
# 프로젝트 구조
- src/ 아래에 주요 코드
- 프레임워크: [Next.js / FastAPI / 등]

# 빌드 명령어
- 빌드: npm run build
- 테스트: npm test

# 규칙
- [에이전트가 실수할 때마다 여기에 추가]
```

Step 2: 피드백 루프 구축하기

가장 간단한 피드백 루프부터 시작합니다:

6. 에이전트가 코드 생성
7. 린터가 자동으로 검사
8. 테스트가 자동으로 실행
9. 실패 시 에이전트가 오류 메시지를 받고 수정
10. 성공하면 조용히 다음 단계로 (성공 출력은 숨김)

Step 3: 점진적 하네스 강화

하네스 구축은 반복적 프로세스입니다. OpenAI도 5개월에 걸쳐 점진적으로 하네스를 발전시켰습니다.

- 1주차: **CLAUDE.md** 기본 작성, 빌드/테스트 명령어 정리
- 2주차: 에이전트 실수 패턴 관찰 → **CLAUDE.md**에 규칙 추가
- 3주차: **Hooks** 설정 (포매팅, 린트 자동화)
- 4주차: **CI/CD** 파이프라인과 연결, 구조 테스트 추가
- 이후: 서브 에이전트, **MCP** 연결, 평가 에이전트 도입

Chapter 13. 비개발자를 위한 하네스 엔지니어링

13.1 비개발자에게 하네스가 중요한 이유

하네스 엔지니어링의 핵심 원리는 코딩에만 적용되는 것이 아닙니다. AI를 활용하는 모든 작업에서 동일한 원리가 적용됩니다:

- 콘텐츠 제작: 블로그, 유튜브 스크립트, SNS 콘텐츠
- 마케팅: 광고 카피, 세일즈 퍼널, 랜딩페이지
- 교육: 강의 자료, 커리큘럼, 학습 콘텐츠
- 비즈니스 운영: 보고서, 분석, 의사결정 지원

13.2 콘텐츠 제작에 하네스 적용하기

예시: 유튜브 대본 제작 하네스

- # YouTube 대본 제작 규칙 (컨텍스트)
 - 타겟: AI에 관심 있는 30대 직장인
 - 톤: 친근하고 실용적인 반말체
 - 구조: Hook(15초) → 문제제기 → 해결책 → CTA
- # 제약조건 (건축적 제약)
 - 한 영상의 핵심 메시지는 1개만
 - 전문용어 사용 시 반드시 즉시 설명
 - 영상 길이: 8분 이내
- # 피드백 루프
 - 1차 초안 → 검토(틀, CTA 유무, 길이) → 수정 → 최종
 - 확인 기준: '이 영상을 본 사람이 바로 실행할 수 있는가?'

13.3 마케팅에 하네스 적용하기

예시: 병원 블로그 자동화 하네스

- # 컨텍스트
 - 병원: [OO피부과], 전문분야: [피부 미용]
 - 타겟 환자: 피부 고민이 있는 30~50대 여성

제약조건

- 의학적 주장에는 반드시 근거 제시
- 과대 광고 표현 금지
- SEO 키워드 반드시 포함

피드백 루프

- 초안 → 의학적 정확성 검증 → SEO 검사 → 톤 조정 → 최종

Chapter 14. 하네스 엔지니어링 체크리스트

실무에서 하네스를 구축할 때 활용할 수 있는 체크리스트입니다:

단계	체크 항목	상태
기초 설정	CLAUDE.md / AGENTS.md 작성	☐
기초 설정	프로젝트 구조 및 빌드 명령어 문서화	☐
기초 설정	코딩 규칙 및 컨벤션 정의	☐
피드백 루프	린터 자동 검사 설정	☐
피드백 루프	테스트 자동 실행 설정	☐
피드백 루프	성공/실패 출력 필터링 (실패만 표시)	☐
훅 설정	Pre-tool hooks 구성 (보안 검사)	☐
훅 설정	Post-tool hooks 구성 (포맷팅)	☐
훅 설정	Stop hooks 구성 (최종 검증)	☐
고도화	서브 에이전트 병렬 처리 구성	☐
고도화	Generator-Evaluator 분리 구성	☐
고도화	MCP 서버 연결 (필요 시)	☐
운영	에이전트 실패 패턴 로깅 및 분석	☐
운영	CLAUDE.md 주기적 업데이트	☐
운영	불필요한 복잡성 제거 (Rip it out)	☐

PART 5

미래 전망과 참고 자료

Chapter 15. 하네스 엔지니어의 미래

15.1 새로운 직무의 탄생

하네스 엔지니어는 새로운 역할입니다. 소프트웨어 엔지니어가 애플리케이션을 만들고, ML 엔지니어가 모델을 만든다면, 하네스 엔지니어는 모델과 실제 세계 사이의 다리를 만듭니다.

필요한 역량: 소프트웨어 엔지니어링 + AI 시스템 이해 + 사용자 경험(UX) 설계 능력

15.2 하네스가 새로운 서비스 템플릿이 된다

Martin Fowler는 하네스가 '새로운 서비스 템플릿'이 될 가능성을 제시했습니다. 특정 애플리케이션 유형에 맞는 사전 구성된 하네스를 선택하고, 이를 커스터마이징하는 방식으로 진화할 수 있습니다.

15.3 내구성(Durability) 문제

Philipp Schmid는 미래의 핵심 병목 현상으로 'Context Durability(컨텍스트 내구성)'을 지목했습니다. 모델이 100번째 도구 호출 이후에도 지침을 일관되게 따를 수 있는지를 하네스가 모니터링하고, 이 데이터를 학습에 피드백하는 흐름이 발전할 것입니다.

15.4 Bitter Lesson과 하네스

모델이 매번 업그레이드될 때마다 어제의 복잡한 하네스 로직이 불필요해질 수 있습니다. Vercel이 에이전트 톨의 80%를 제거하고 성능이 향상된 것처럼, 하네스는 항상 가볍게 유지하고 쉽게 탈부착할 수 있어야 합니다.

Chapter 16. 핵심 용어 사전

용어	정의
Harness	에이전트를 둘러싼 제약, 톨, 피드백, 관찰 시스템의 총체
Harness Engineering	하네스를 설계·구축·운영하는 기술 분야
Context Engineering	모델에게 적절한 정보를 적절한 시점에 제공하는 기술
CLAUDE.md / AGENTS.md	프로젝트의 규칙과 문맥을 담은 설정 파일
Hooks	에이전트 생명주기의 특정 시점에 자동 실행되는 스크립트
Skills	필요 시점에만 로딩되는 점진적 정보 공개 메커니즘
Sub-agent	주 에이전트 아래에서 별도 컨텍스트로 작동하는 하위 에이전트
MCP	Model Context Protocol . 에이전트와 외부 서비스를 연결하는 표준
Context Anxiety	컨텍스트 윈도우 안에서 에이전트 성능이 하락하는 현상
Context Reset	에이전트에게 깨끗한 컨텍스트로 재시작을 제공하는 방법
Generator-Evaluator	생성과 평가를 분리한 듀얼 에이전트 구조
Rippable Harness	모델 업그레이드 시 쉽게 제거할 수 있는 경량 하네스
Durability	에이전트가 수백 번의 도구 호출 후에도 일관성을 유지하는 능력

Chapter 17. 참고 자료 및 추천 리소스

17.1 핵심 원문 (Primary Sources)

11. Mitchell Hashimoto, 'My AI Adoption Journey' (2026.02.05) — mitchellh.com
12. OpenAI, 'Harness engineering: leveraging Codex in an agent-first world' (2026.02) — openai.com
13. Anthropic, 'Effective harnesses for long-running agents' (2025.11) — anthropic.com/engineering
14. Anthropic, 'Harness design for long-running application development' (2026) — anthropic.com/engineering
15. Philipp Schmid, 'The importance of Agent Harness in 2026' (2026.01) — philschmid.de
16. Birgitta Böckeler / Martin Fowler, 'Harness Engineering' (2026.02.17) — martinfowler.com

17.2 해설 및 분석 자료 (Secondary Sources)

17. Cobus Greyling, 'The Rise of AI Harness Engineering' (2026.03) — Medium
18. Louis Bouchard, 'Harness Engineering: The Missing Layer Behind AI Agents' (2026.03) — louisbouchard.ai
19. SmartScope, 'What Is Harness Engineering' (2026.02, 2026.03 개정) — smartscope.blog
20. Aakash Gupta, '2025 Was Agents. 2026 Is Agent Harnesses.' (2026.01) — Medium
21. HumanLayer, 'Skill Issue: Harness Engineering for Coding Agents' (2026.03) — humanlayer.dev
22. Epsilla, 'The Third Evolution: Why Harness Engineering Replaced Prompting in 2026' (2026.03) — epsilla.com

17.3 도구 및 플랫폼

23. Claude Code: code.claude.com/docs
24. OpenAI Codex: openai.com/codex
25. LangChain / LangGraph: langchain.com
26. CrewAI: crewai.com
27. Cursor: cursor.com

하네스 엔지니어링 완전 가이드북

에이나우(AINOW) | 2026년 3월

본 전자책은 공개된 자료, 연구 결과, 기업 발표 내용을 기반으로 구성되었습니다.

AI 시대의 생산자를 위한 실전 지침서